PROJECT INITIATION DOCUMENT

# JailGuard API

AI-Powered Multi-Turn Jailbreak Detection

| | |
|---|---|
| **Project Owner** | Yash |
| **Version** | 1.0 — Initial |
| **Date** | 6 June 2026 |
| **Status** | Draft |
| **Timeline** | 3 Months (Full Commitment) |

# 1. Executive Summary

JailGuard is a real-time REST API that detects jailbreak attacks in AI chatbot conversations. Unlike existing tools that analyse only a single message, JailGuard uses a sliding-window approach to monitor entire conversation histories — catching multi-turn manipulation attempts that unfold gradually across several messages.

| | |
|---|---|
| **Core Idea** | Developers building on top of ChatGPT, Claude, or any LLM drop one API call into their app. Before a user message reaches the AI, JailGuard checks the full conversation context and returns a risk score in under 200ms. |

# 2. Problem Statement

## 2.1 What is a jailbreak?

A jailbreak is an attempt by a user to manipulate an AI model into ignoring its safety guidelines — for example, generating harmful content, revealing confidential system prompts, or bypassing ethical restrictions. These attacks are increasingly sophisticated and common.

## 2.2 Why existing tools fall short

Current moderation tools, including OpenAI's Moderation API, evaluate messages in isolation. They answer: 'Is this single message harmful?' But attackers have learned to spread their attacks across multiple turns — each message looks innocent on its own, but together they steer the AI toward a harmful outcome.

| Limitation | Current Tools | JailGuard |
|---|---|---|
| Multi-turn awareness | ✗ Single message only | ☑ Full conversation window |
| Attack type classification | ✗ Binary flag only | ☑ Labels attack category |
| Self-hostable | ✗ Vendor lock-in | ☑ Open core model |
| LLM-agnostic | ☑ | ☑ |

## 2.3 Who is affected?

- Companies deploying AI customer service bots
- EdTech platforms using AI tutors
- Healthcare apps with AI assistants
- Any developer building a product on top of a large language model

# 3. Project Objectives

## 3.1 Primary objectives

- Build and deploy a production-ready REST API for real-time jailbreak detection
- Achieve sub-200ms latency per inference call
- Support both stateless (full history) and stateful (session-based) usage
- Classify attack type — not just flag jailbreaks, but explain what kind

## 3.2 Secondary objectives

- Publish Python and JavaScript SDKs for easy developer integration
- Build a developer dashboard with usage analytics and flag logs
- Explore commercialisation through a freemium API pricing model
- Contribute findings back to the AI safety research community

## 3.3 Success metrics

| Metric | Target |
|---|---|
| F1 Score on held-out test set | ≥ 0.88 |
| API response latency (p95) | < 200ms |
| API uptime | 99.5% |
| Supported attack types | ≥ 6 categories |
| Developer dashboard | Fully functional at launch |

# 4. Motivation & Rationale

## 4.1 Market opportunity

The global AI safety and governance market is growing rapidly alongside LLM adoption. Every company building AI-powered products faces liability if their system is manipulated into producing harmful outputs. There is currently no widely adopted, multi-turn-aware jailbreak detection tool available as a simple API.

## 4.2 Research foundation

This project is built directly on top of completed academic research — a full research paper on multi-turn jailbreak detection using a sliding-window DistilBERT classifier (SW-DistilBERT). The model has been trained, evaluated across multiple datasets, and validated with real

experimental results. The project phase is now about productionising that research into a deployable, usable tool.

| Key Advantage | The research moat is already built. SW-DistilBERT's sliding window approach demonstrably outperforms single-turn classifiers on multi-turn attack datasets — this is the core differentiator. |
|---|---|

### 4.3 Personal motivation

- Apply NLP research skills to a real, high-impact problem
- Build a portfolio-worthy, deployable AI product from scratch
- Explore a genuine commercial opportunity in the AI security space
- Contribute to safer AI deployment practices

## 5. Technical Approach

### 5.1 System overview

JailGuard is a three-layer system: (1) a fine-tuned DistilBERT classifier as the intelligence core, (2) a FastAPI service that handles the sliding window logic, session state, and response formatting, and (3) an AWS-hosted infrastructure layer for deployment, auth, and scaling.

### 5.2 Technology stack

| Layer | Technology | Purpose |
|---|---|---|
| Model | HuggingFace DistilBERT | Core jailbreak classifier |
| Inference | ONNX Runtime (optional) | 3–4× speed boost via quantisation |
| API | FastAPI (Python) | Async REST API, OpenAPI docs |
| Session state | Redis | Sliding window conversation memory |
| Database | AWS DynamoDB | API keys, usage logs |
| Hosting | AWS ECS + Fargate | Containerised, auto-scaling |
| CDN / Security | AWS CloudFront | DDoS protection, caching |
| Containerisation | Docker | Reproducible deployment |
| SDKs | Python + JavaScript | Developer integration packages |

### 5.3 Core API contract

POST /v1/analyze — Accepts a session ID and list of conversation messages. Returns a risk score (0–1), a classification label (safe / suspicious / jailbreak), the attack type, which turn was flagged, and per-turn window scores. This gives developers both a simple binary signal and fine-grained diagnostic information.

# 6. Project Plan

## 6.1 Phase breakdown

| Phase | Weeks | Focus | Key Deliverables |
|---|---|---|---|
| 1 — Model | 1–3 | ML / Research | Exported weights, calibration layer, extended dataset training, multi-label output, benchmark report |
| 2 — API | 4–6 | Backend | FastAPI service, /v1/analyze endpoint, Redis session integration, API key auth, unit tests |
| 3 — Infra | 7–9 | DevOps / Cloud | Docker image, AWS ECS deployment, CloudFront, monitoring (CloudWatch), load testing |
| 4 — Product | 10–12 | Product / GTM | Developer dashboard, Python + JS SDKs, pricing page, documentation, public launch |

## 6.2 Week-by-week breakdown

**Phase 1 — Model (Weeks 1–3)**

- Week 1: Export SW-DistilBERT weights, set up GitHub repo, establish benchmarking pipeline
- Week 2: Add confidence calibration (Platt scaling), train on JailbreakBench + WildChat datasets
- Week 3: Implement multi-label classification output (6+ attack types), write benchmark report

**Phase 2 — API (Weeks 4–6)**

- Week 4: Scaffold FastAPI project, implement /v1/analyze endpoint, sliding window logic
- Week 5: Integrate Redis for session state, add API key authentication middleware
- Week 6: Write 20+ integration tests, optimise for <200ms latency, internal load testing

**Phase 3 — Infrastructure (Weeks 7–9)**

- Week 7: Dockerise the service, set up CI/CD pipeline (GitHub Actions)
- Week 8: Deploy to AWS ECS + Fargate, configure CloudFront, set up DynamoDB

- Week 9: Monitoring dashboards (CloudWatch), security audit, stress testing

**Phase 4 — Product (Weeks 10–12)**

- Week 10: Build developer dashboard (React), usage charts, flag logs
- Week 11: Publish Python SDK (PyPI) and JS SDK (npm), write developer documentation
- Week 12: Set up pricing tiers, write launch blog post, submit to HackerNews / Product Hunt

## 7. Risks & Mitigations

| Risk | Severity | Mitigation |
|---|---|---|
| Model accuracy drops on new jailbreak styles | High | Continuous retraining pipeline; monitor JailbreakBench for new attack patterns quarterly |
| Latency exceeds 200ms threshold | Medium | ONNX Runtime quantisation; GPU instance upgrade; response caching for repeated patterns |
| API abuse / adversarial scraping | Medium | Rate limiting per API key; anomaly detection on usage patterns; tiered access controls |
| Low developer adoption | Medium | Free tier with 1,000 calls/month; one-line integration; clear documentation and demo |
| Cloud costs exceed budget | Low | Start on single EC2 instance; only move to ECS when traffic justifies; set billing alerts |

## 8. Motive Summary

| | |
|---|---|
| **What** | A REST API that detects multi-turn jailbreak attacks in LLM-powered applications in real time. |
| **Why** | AI products are being manipulated at scale. Existing tools only look at single messages. Multi-turn attacks are undetected. This is a real, growing, unsolved problem. |
| **How** | SW-DistilBERT classifier wrapped in a FastAPI service, deployed on AWS, with session state via Redis and developer tooling (SDKs + dashboard). |

| | |
|---|---|
| **Who** | Any developer or company building products on top of LLMs — from startups to enterprise — who needs a drop-in safety layer. |

| | |
|---|---|
| **Goal** | Launch a production API in 12 weeks. Achieve 88%+ F1 accuracy. Sub-200ms latency. Free tier to drive adoption, paid tiers for sustainability. |